Health Symptom Analyzer

Dax Rajani (ID: 40294118), Harsh Ahuja (ID: 40301748), Charanish Miriyala (ID: 40307568)
Department of Electrical and Computer Engineering
Concordia University, Montréal, Canada
Emails: daxrajani@gmail.com, ahujaharsh20@gmail.com, charanish005@gmail.com

***Abstract*—Early detection of disease often leads to better treatment options, fewer complications, and lower overall healthcare costs. In real life, however, people frequently ignore early symptoms, underestimate how serious they might be, or are not able to see a doctor quickly. This creates room for simple decision-support tools based on machine learning (ML) that convert self-reported symptoms into a rough, non-diagnostic indication of which conditions might be plausible, nudging users toward expert care when needed.**

In this work, we build and evaluate a complete ML pipeline for disease prediction using a carefully selected disease–symptom dataset. We begin from a UMLS-derived knowledge source containing 134 diseases and 409 coded clinical concepts and transform it into a prototype dataset with 41 common diseases, 132 binary symptom attributes, and 4 920 synthetic patient records. We perform exploratory data analysis (EDA), design a modular preprocessing and training framework, and test eight supervised classifiers: Naive Bayes, Decision Tree, Random Forest, Support Vector Machine (SVM), k-Nearest Neighbours (kNN), Logistic Regression, XGBoost, and a Voting Ensemble. After tuning hyperparameters, we examine both quantitative performance and qualitative error patterns.

Our experiments show that linear models, in particular Logistic Regression and SVMs, perform extremely well on the prototype dataset, achieving 99.39% accuracy and an MCC of 0.9938, which suggests that the binary symptom space is almost linearly separable. Ensemble methods such as Random Forest, XGBoost, and a Voting Classifier provide additional stability, whereas a single Decision Tree is highly volatile and performs much worse (83.94% accuracy). A closer look at misclassified examples shows that most mistakes occur between diseases with very similar symptom profiles, such as liver disorders, inflammatory conditions, and certain opportunistic infections. We conclude with a discussion of practical deployment issues, limitations of relying only on binary symptom encodings, and possible extensions including hierarchical classification, richer feature sets, and integration into clinical workflows.

***Index Terms*—Disease prediction, machine learning, healthcare AI, classification, ensemble methods, exploratory data analysis, symptoms.**

I. Introduction

A. Motivation

Early detection of medical problems is closely linked to improved patient outcomes, fewer complications, and reduced long-term healthcare costs. In practice, though, many people ignore minor or early-stage symptoms, postpone seeing a doctor, or simply cannot access a clinician in time. These gaps motivate the use of ML-based decision-support tools that can turn basic self-reported symptom information into rough guidance about likely diseases and help individuals decide whether they should seek professional care.

B. Diagnostic Challenges

From a clinical point of view, working solely from symptoms is challenging for at least two reasons. First, many symptoms are non-specific and can occur in a wide range of illnesses. Common complaints such as fever or fatigue may appear in dozens of infectious, inflammatory, or malignant conditions, making it hard to determine the underlying disease when only a few symptoms are known. Second, the information available at the first encounter is usually incomplete: patients often mention only a subset of the symptoms they experience. This leads to high-dimensional but very sparse input vectors, which makes manual reasoning difficult. These characteristics naturally suggest computational methods that can detect subtle patterns in the joint distribution of diseases and symptoms.

C. Machine Learning Objective

Our machine learning goal is to treat disease prediction as a supervised multi-class classification problem. Each patient is represented by a binary symptom vector $\mathbf{x} \in \{0,1\}^{132}$, where each component indicates whether a particular symptom (such as itching, nausea, or high fever) is present or absent. The model produces a probability distribution over 41 disease classes,

$$\mathbf{p} = \big(P(C_1 \mid \mathbf{x}), \ldots, P(C_{41} \mid \mathbf{x})\big),$$

from which we extract the top-k most likely diagnoses. We aim to maximise test accuracy, macro-averaged precision, recall, and F1-score, while keeping false negatives for serious conditions as low as possible. We also require that the trained system be fast enough for interactive use and transparent enough that clinicians can see which symptoms contribute most to each prediction.

D. Research Question and Scope

The primary question we seek to answer is: *How accurately can conventional supervised ML models, trained exclusively on binary symptom presence or absence, predict the most probable disease, and what does a complete, end-to-end system for this objective need to look like to support reproducibility and in-depth analysis?*

To keep the problem clear and manageable, we intentionally restrict ourselves to symptom-only inputs. This allows us to:

- quantify how much predictive power symptoms alone can provide,
- compare several standard ML methods on the same dataset under consistent conditions,

- highlight limitations that must be addressed before considering deployment in real clinical settings.

E. Contributions

The main contributions of this work are:

1) A modular, end-to-end ML pipeline for symptom-based disease prediction, including data generation, exploratory data analysis (EDA), preprocessing, model training, evaluation, and interpretability.
2) A comparative study of eight supervised ML models on a multi-class disease prediction task using binary symptom vectors as input.
3) A qualitative error analysis linking misclassifications to clinically similar diseases that share overlapping symptom patterns.
4) A two-branch implementation (model-development and model-evaluation) that separates interactive prediction from offline experimentation, with reusable utilities for data loading and model construction.

II. Background and Related Work

A. Disease–Symptom Knowledge Bases

Structured knowledge bases that capture relationships between diseases and symptoms are central to many decision-support tools. The Disease–Symptom Knowledge Database (DSKB) from Columbia University is one such resource: it uses the MedLEE NLP system to extract Unified Medical Language System (UMLS) concepts from discharge summaries and aggregates them into weighted disease–symptom associations [6], [7]. The public description stresses that these relationships are derived automatically from real clinical text rather than specified manually by experts [5]. In our project, the raw data file `data_generation/main.csv` is a curated subset of such a knowledge base.

B. ML-Based Disease Prediction

There is extensive research on ML methods for early disease diagnosis, covering conditions such as diabetes, cardiovascular disease, and various communicable diseases [8], [9]. In recent years, several studies have focused specifically on symptom-based multi-class disease prediction and report accuracies in the 90–97% range on real or synthetic datasets [10]. These systems typically use models such as Logistic Regression, Random Forest, Gradient Boosting, or neural networks, and often emphasise interpretability and clinical validation.

C. Supervised Algorithms in Healthcare

Comparative analyses of supervised learning methods for disease risk prediction often find that Logistic Regression and SVMs are strong starting points and can compete with more complex models, especially when feature engineering is limited and datasets are of moderate size [9]. Tree-based ensembles such as Random Forest and Gradient Boosting (including XGBoost [11]) can sometimes achieve slightly higher accuracy, but they are usually harder to interpret, which is a concern in clinical settings.

D. Explainable AI in Clinical Settings

Explainability is widely viewed as a key requirement for clinical use of ML-based decision support. SHAP (SHapley Additive exPlanations) attributes model predictions to individual features using Shapley values from cooperative game theory and has been applied in many healthcare applications [12]–[14]. LIME (Local Interpretable Model-Agnostic Explanations) fits simple local surrogate models to explain why a given black-box classifier makes a particular prediction [15], [16]. In our work, we use both SHAP and LIME to provide global and local explanations for tree-based and ensemble models.

III. Dataset and Problem Formulation

A. From Raw UMLS Data to Prototype Dataset

The original file `data_generation/main.csv` contains 134 diseases and 409 encoded symptom columns, along with a frequency column indicating how often each disease–symptom pattern occurs. The script `data_generation/generate_prototype.py` converts this into a more convenient prototype dataset by:

- renaming UMLS-style column headers such as `UMLS:C0018681_headache` to simpler names like `headache`,
- mapping disease codes in the `label` column to human-readable disease names stored in a new `prognosis` column,
- selecting a subset of clinically meaningful symptoms that are sufficiently frequent,
- generating synthetic patient records by sampling symptom configurations according to the observed frequency distribution.

The resulting file, `Prototype.csv`, has:

- 4 920 rows (120 synthetic patient records per disease),
- 132 binary symptom features,
- one categorical label column, `prognosis`, representing 41 distinct diseases.

B. Symptom Vocabulary

For deployment, the list of supported symptoms is stored in `available_symptoms.txt`. The interactive script `main.py` reads this file to validate user input and construct input vectors in the same feature order that was used during training and evaluation.

C. Multi-Class Classification Task

Using the prototype dataset, the main prediction problem can be formalised as a multi-class classification task:

- **Input:** a binary vector $\mathbf{x} \in \{0,1\}^{132}$, where $x_j = 1$ if symptom j is reported and 0 otherwise.
- **Output:** a label $y \in \{1, \ldots, 41\}$ indicating the predicted disease.

The model learns a mapping $f(\mathbf{x}) \approx y$ from the training data and is then evaluated on unseen symptom patterns from the test set.

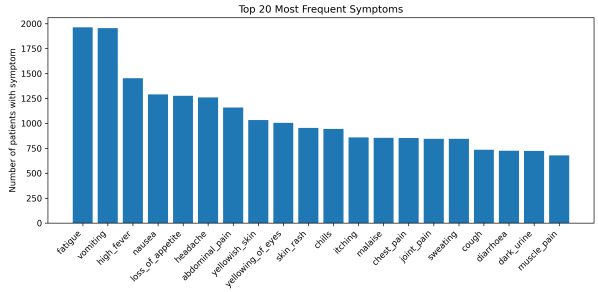

Fig. 1. Top 20 most frequent symptoms in the prototype dataset

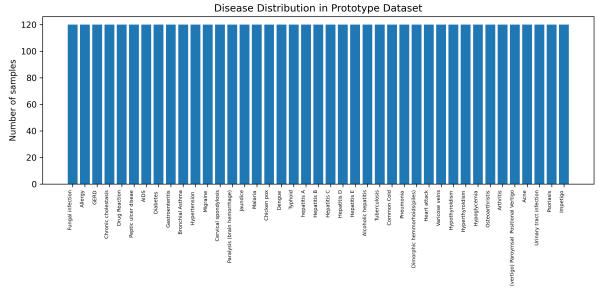

Fig. 2. Disease distribution in the prototype dataset

D. Intended Use-Case

The system is meant to be a decision-support and teaching tool, not a diagnostic or triage system. Its goals are to:

- produce a rough ranked list of diseases that might be consistent with a given set of self-reported symptoms,
- encourage users to see a clinician when the model suggests potentially serious conditions,
- serve as a concrete example for teaching applied ML methods in a healthcare setting.

IV. Exploratory Data Analysis

A. Symptom Frequency Distribution

An exploratory analysis of the original knowledge base shows that a relatively small set of symptoms—including shortness of breath, fever, fatigue, and several types of pain—occur very frequently across diseases. In the prototype dataset, we see a similar pattern: symptoms such as `vomiting`, `fatigue`, `high_fever`, and `nausea` appear often, providing strong but non-specific signals.

B. Disease Distribution

By construction, the prototype dataset contains exactly 120 instances of each of the 41 diseases. This results in a perfectly balanced label distribution, which simplifies model training, evaluation, and interpretation.

C. Symptoms per Patient

Fig. 3 shows how many active symptoms each synthetic patient has. Most records contain between 4 and 10 symptoms, with fewer cases outside that range. This resembles realistic

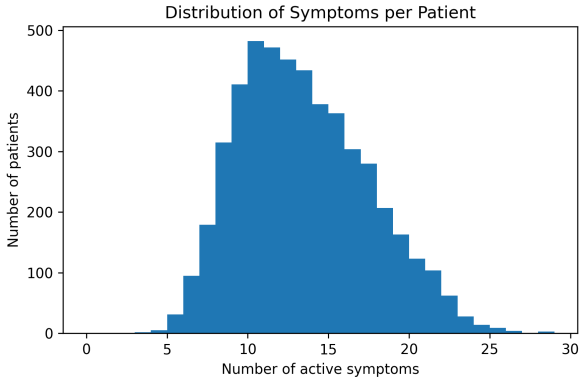

Fig. 3. Distribution of the number of active symptoms per sample

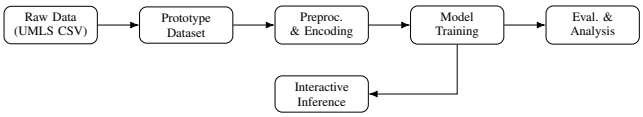

Fig. 4. High-level system architecture for symptom-based disease prediction

clinical scenarios, where patients rarely present with an extremely long list of complaints all at once.

D. Disease–Symptom Co-Occurrence

Heatmaps of disease–symptom co-occurrence for both the original and prototype datasets show clusters of diseases that share many systemic symptoms, as well as an overall sparse structure where most entries are zero for a given disease. Symptom co-occurrence matrices exhibit relatively low pairwise correlations, which is advantageous for models that assume conditional independence (such as Naive Bayes) and helps explain why linear models can work effectively in this setting.

V. System Architecture

A. High-Level Architecture

The system follows a modular, data-centric design that separates data generation, preprocessing, model training, offline evaluation, and interactive use. The first step is to convert raw disease–symptom associations from the UMLS-based knowledge base into the balanced prototype dataset. This dataset is then transformed into binary symptom vectors and used to train several supervised learning models. The trained models support two main workflows: batch evaluation (metrics, confusion matrices, and interpretability plots) and interactive prediction through a command-line interface, as summarised in Fig. 4.

B. Data-Centric Pipeline

The implementation breaks the workflow into a series of reusable stages, shown in Fig. 5. The raw dataset is first converted into the prototype dataset and explored using EDA. The data are then split into training and test sets. Model training and cross-validated hyperparameter tuning take place on the training data, and the best-performing configurations are

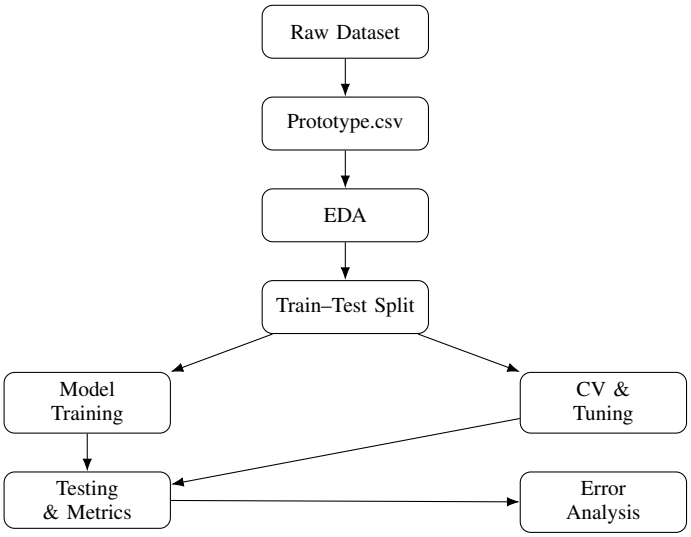

Fig. 5. End-to-end pipeline for model development and evaluation

evaluated on the held-out test set. Additional scripts generate summary metrics, confusion matrices, and interpretability artefacts for further examination.

VI. Methods

A. Preprocessing

The evaluation script `scripts/evaluation.py` performs the following preprocessing:

- an 80/20 train–test split using `train_test_split` with stratification by disease,
- label encoding of disease names stored in the `prognosis` column,
- enforcement of a consistent feature order across training, evaluation, and interactive inference.

B. Data Loading and Train–Test Protocol

In the development branch, `data_loader.py` provides a general interface for loading training and test data from specified CSV files (for example, `Prototype-1.csv` for training and `Prototype.csv` for testing), given explicit lists of symptoms and diseases. It:

- builds a mapping from disease names to numeric labels based on the given disease list,
- drops any rows whose diseases are not present in the mapping,
- splits features and labels into NumPy arrays,
- ensures that training and test sets share the same feature ordering and label encoding.

This design allows different dataset variants to be plugged into the same modelling code with only small adjustments.

C. Feature Representation

We use the full 132-dimensional binary symptom vector as the feature representation, without applying dimensionality reduction. This keeps a one-to-one correspondence between feature dimensions and named symptoms, which is useful for interpretability.

Model	Capacity	Interpretability
Naive Bayes	Low	High
Logistic Regression	Low–Medium	High
SVM (RBF)	Medium	Medium
kNN	Medium	Medium–Low
Decision Tree	Medium	Medium–High
Random Forest	Medium–High	Medium
XGBoost	High	Medium–Low
Voting Ensemble	High	Medium

Fig. 6. Qualitative comparison of model families by capacity and interpretability

D. Model Families and Hyperparameters

Model definitions in `models/*.py` (development and evaluation branches) specify tuned hyperparameters, including:

- Logistic Regression: $C = 0.1$, L2 penalty, `liblinear` solver, `max_iter = 1000`,
- SVM: RBF kernel, $\gamma = 0.001$, $C = 1$, `probability = True`,
- kNN: $k = 11$ neighbours, distance-based weighting,
- Random Forest: 100 trees, maximum depth 20, `min_samples_leaf = 4`,
- Decision Tree: `criterion='entropy'`, maximum depth 20,
- Naive Bayes: Gaussian NB with `var_smoothing = 1.0`,
- XGBoost: 200 estimators, learning rate 0.1, maximum depth 6, subsample 0.7 [11].

A Voting Classifier in `ml_models.py` combines multiple base learners using both soft and hard voting schemes.

E. Hyperparameter Tuning

The notebook `scripts/tune_all_models.ipynb` performs grid search over selected hyperparameters for each model family, optimising for validation accuracy and macro F1-score. The best configurations are saved to disk and reused by the main evaluation script and the interactive prediction tool.

VII. Mathematical Formulation

A. Logistic Regression

For multi-class Logistic Regression with classes $k \in \{1, \ldots, K\}$ and feature vector $\mathbf{x}$, we model:

$$P(y = k \mid \mathbf{x}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x} + b_k)}{\sum_{j=1}^{K} \exp(\mathbf{w}_j^\top \mathbf{x} + b_j)}. \tag{1}$$

The parameters $\{\mathbf{w}_k, b_k\}$ are learned by minimising cross-entropy loss with L2 regularisation.

B. Support Vector Machine

For the SVM with an RBF kernel, the decision function for class k can be written as:

$$f_k(\mathbf{x}) = \sum_{i=1}^{N} \alpha_{ik} K(\mathbf{x}_i, \mathbf{x}) + b_k, \tag{2}$$

where $K(\mathbf{x}_i, \mathbf{x}) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}\|^2)$ is the kernel function and α_{ik} are the learned dual coefficients subject to margin maximisation constraints.

C. Naive Bayes

Naive Bayes assumes that symptoms are conditionally independent given the disease label:

$$P(y = k \mid \mathbf{x}) \propto P(y = k) \prod_{j=1}^{d} P(x_j \mid y = k). \tag{3}$$

In our experiments, Gaussian NB serves as a simple baseline model.

D. Evaluation Metrics

For true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN), the standard metrics are defined per class and then averaged macro-wise:

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{4}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \tag{5}$$

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{6}$$

The Matthews Correlation Coefficient (MCC) is computed from the confusion matrix in its multi-class form and provides a single scalar summary of performance that remains meaningful even when classes are imbalanced.

VIII. Experimental Setup

A. Train–Test Split

All models are evaluated using an 80/20 train–test split of the prototype dataset:

- 3 936 samples for training,
- 984 samples for testing,
- stratification by disease so that label distributions remain identical in both sets.

Random seeds are fixed in the evaluation script to ensure reproducible results.

B. Baselines and Tuned Models

For each algorithm, we examine both a default baseline configuration and a tuned configuration obtained via grid search. The comparative results reported in the next section correspond to the tuned models stored in `saved_models_main/`.

TABLE I
Model performance comparison on prototype test set

Model	Accuracy	F1-score	MCC
Logistic Regression	0.9939	0.9939	0.9938
SVM	0.9939	0.9939	0.9938
XGBoost	0.9929	0.9929	0.9927
kNN	0.9919	0.9919	0.9917
Naive Bayes	0.9919	0.9918	0.9917
Voting Classifier	0.9919	0.9918	0.9917
Random Forest	0.9888	0.9889	0.9886
Decision Tree	0.8394	0.8400	0.8357

C. Interactive CLI Evaluation

The development branch script `main.py` implements a simple interactive command-line interface that:

- loads disease and symptom names from the prototype dataset,
- loads trained models from `saved_models_main/`,
- prompts the user to enter a list of symptoms,
- validates these symptoms against `available_symptoms.txt`,
- builds the corresponding binary input vector and prints predictions from each model, as well as soft and hard voting outputs.

This demonstrates that the trained models are lightweight enough for real-time interaction on standard hardware.

D. Computing Environment

All experiments are run on a laptop-class CPU. Given the modest dataset size and model complexity, training for each model completes within a few seconds, supporting the feasibility of similar pipelines in resource-constrained environments.

IX. Results

A. Overall Model Comparison

Table I compares the performance of all models on the test set, based on the metrics stored in `evaluation_results/comparative_metrics.csv`.

Logistic Regression and SVM achieve the best overall performance, with XGBoost and kNN close behind. Naive Bayes and the Voting Classifier also perform strongly, while Random Forest is slightly weaker. The single Decision Tree is the clear outlier, with much lower accuracy and MCC, reflecting its higher variance and sensitivity to particular splits.

B. Confusion Matrix

Fig. 7 shows the normalised confusion matrix for Logistic Regression on the test set. Most of the mass lies on the main diagonal, indicating correct predictions, while the smaller off-diagonal entries highlight a few disease pairs that are commonly confused.

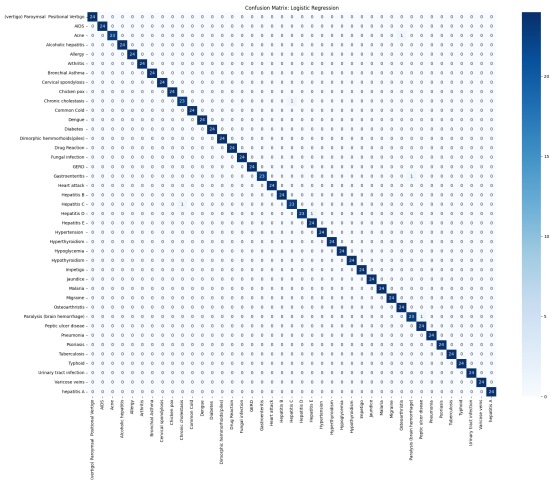

Fig. 7. Confusion matrix of Logistic Regression on the test set

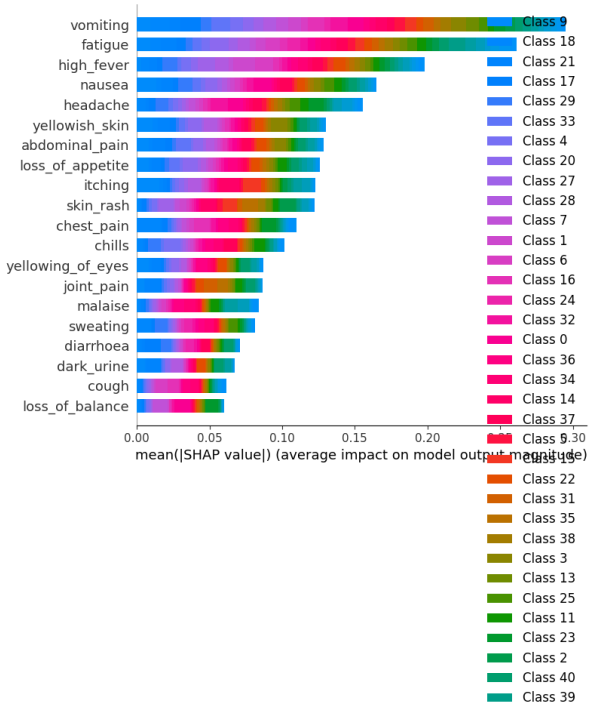

Fig. 8. SHAP summary plot showing global feature importance for the XGBoost model

C. Feature Importance via SHAP

For the XGBoost model, we use SHAP to estimate global feature importance. The summary plot in Fig. 8 shows which symptoms have the strongest influence on predictions. Symptoms such as `high_fever`, `vomiting`, `abdominal_pain`, `yellowish_skin`, and `fatigue` appear among the top contributors, which is consistent with medical intuition.

X. Case Studies and Error Analysis

A. Misclassified Examples

The file `evaluation_results/error_analysis.txt` reports eight misclassified test instances out of 984. Typical examples include:

- Hepatitis C predicted as chronic cholestasis,
- Gastroenteritis predicted as arthritis,
- Fungal infection predicted as an AIDS-related condition.

For each case, the script logs the full list of symptoms, allowing detailed inspection of why the classifier may have been uncertain.

B. Clinical Ambiguity

These confusions are mostly clinically plausible. Many liver diseases share symptoms such as jaundice, fatigue, and abdominal pain; inflammatory and gastrointestinal conditions often present with overlapping pain and systemic symptoms. Without laboratory tests, imaging, or temporal information, even experienced clinicians may find it difficult to distinguish between these possibilities when only binary symptom presence is available.

C. Local Explanations with LIME

For two illustrative examples (one correctly classified and one misclassified), we generate LIME explanations and save them as HTML files in `evaluation_results/interpretability`. LIME highlights the symptoms that most strongly support the predicted disease label, providing a local, human-readable approximation of the classifier's decision boundary around each case.

XI. Implementation Details and Reproducibility

A. Repository Structure

The project repository is organised as follows:

- **Root:** `Prototype.csv`, `available_symptoms.txt`, `main.py`, `requirements.txt`,
- **data_generation/**: raw dataset and prototype generation script,
- **models/**: model wrapper files for each algorithm (development and evaluation variants),
- **scripts/**: evaluation and hyperparameter tuning scripts/notebooks,
- **saved_models_main/**: pre-trained models for interactive use,
- **evaluation_results/**: metrics tables, per-model reports, confusion matrices, and interpretability outputs.

B. Interactive Prediction

The script `main.py`:

- loads the prototype dataset to recover label encodings,
- loads trained models from `saved_models_main/`,
- prompts the user for symptom input and validates each entry against `available_symptoms.txt`,

- constructs the corresponding binary feature vector and returns predictions from each individual model and from the Voting Classifier.

If the user provides only a very small number of symptoms, the script displays a warning that predictions may be unreliable and encourages users to seek medical attention for concerning issues.

C. Environment

The `requirements.txt` file lists the Python packages needed to reproduce the environment, including `scikit-learn`, `xgboost`, `pandas`, `numpy`, `matplotlib`, `seaborn`, `shap`, and `lime`.

XII. Discussion

A. Impact of Binary Symptom Encoding

Binary symptom encoding simplifies the feature space and works well with many classical algorithms, but it ignores important details such as severity, duration, and temporal progression. The small number of remaining misclassifications is likely due more to this loss of nuance than to fundamental limitations of the learning algorithms.

B. Model Choice and Interpretability

In our setting, linear models perform very well while remaining easy to interpret. Tree-based and ensemble models capture more complex non-linear interactions but are less transparent. In line with earlier work on ML-based disease prediction [8], [9], our results suggest that well-regularised linear models are strong baselines for structured symptom datasets that are relatively clean and well organised.

C. Explainability in Clinical Workflows

Methods such as SHAP and LIME can help make model predictions more understandable for clinicians by clearly showing which features drive each decision [13], [14]. However, the practical value of these explanations depends on how they are presented and how they fit into clinical workflows. Our project demonstrates the technical integration of SHAP and LIME; evaluating their actual impact on trust, usability, and decision-making would require user studies involving clinicians and possibly patients.

D. Threats to Validity

This work has several limitations:

- The prototype dataset is synthetic and perfectly balanced, which does not reflect real clinical populations.
- Biases in the original clinical notes and UMLS coding may propagate into the derived dataset.
- The models have not been tested on external, real-world patient data.

Because of these factors, the reported performance metrics should not be taken as evidence that the models are ready for clinical deployment.

XIII. Future Work

A. Hierarchical and Multi-Stage Classification

A natural extension is to explore hierarchical or multi-stage classifiers that first predict a general disease category and then refine the prediction to a more specific condition. This more closely matches how clinicians reason, starting with a broad differential diagnosis and narrowing it down as additional information becomes available.

B. Feature Enrichment

Future versions of the system could incorporate richer feature sets, such as:

- continuous clinical measurements (vital signs, laboratory values),
- demographic variables and known risk factors,
- temporal information about symptom onset and evolution.

Combining symptom data with these additional signals may significantly improve predictive accuracy and reduce ambiguity among clinically similar diseases.

C. MLOps and Deployment

Real-world deployment would require robust MLOps infrastructure, including data ingestion pipelines, monitoring of model performance, periodic retraining, and strong privacy and security protections for user data. Fairness and bias mitigation would also be crucial, particularly if the system is used across diverse populations and healthcare settings.

XIV. Conclusion

We presented a complete ML framework for predicting diseases from binary symptom data, backed by a reproducible GitHub implementation that spans data generation, model training, evaluation, and interpretability. Using a prototype dataset with 41 diseases, 132 symptom features, and 4 920 synthetic patient records, we showed that classical models—especially Logistic Regression and SVMs—can achieve near-perfect accuracy and MCC on this task, while ensemble methods add extra robustness. At the same time, we emphasised the limitations of symptom-only models and the need for richer clinical information and careful validation before real-world use. Overall, this work offers a practical template for future research on ML-based tools for early disease awareness and healthcare decision support.

References

[1] Exploratory Data Analysis of Disease–Symptom Dataset for Predictive Modeling. User-uploaded project paper, Applied Machine Learning – Fall 2025.

[2] Predictive Modeling of Diseases from Patient-Reported Symptoms. User-uploaded project paper, Applied Machine Learning – Fall 2025.

[3] Applied Machine Learning – Project Team Formation. User-uploaded project document, Fall 2025.

[4] Model Development and Evaluation for Disease Prediction. User-uploaded project paper, Applied Machine Learning – Fall 2025.

[5] Disease–Symptom Knowledge Database (DSKB), Columbia University Dept. of Biomedical Informatics. [Online]. Available: https://people.dbmi.columbia.edu/~friedma/Projects/DiseaseSymptomKB/index.html

[6] X. Wang, M. Friedman, G. Hripcsak, "Automated knowledge acquisition from clinical narrative reports," *J. Am. Med. Inform. Assoc.*, vol. 15, no. 5, pp. 536–543, 2008.
[7] X. Wang *et al.*, "Characterizing environmental and phenotypic associations in patient records," *BMC Bioinformatics*, vol. 10, Suppl 9, S13, 2009.
[8] M. Ahsan *et al.*, "Machine-learning-based disease diagnosis: A comprehensive review," *Healthcare*, 2022.
[9] S. Uddin *et al.*, "Comparing different supervised machine learning algorithms for disease prediction," *BMC Med. Inform. Decis. Mak.*, 2019.
[10] S. Sharma *et al.*, "Machine learning-based symptom–disease prediction: A comprehensive analysis of multi-class classification models in healthcare decision support systems," 2025.
[11] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. KDD*, 2016.
[12] S. Lundberg and S. Lee, "A unified approach to interpreting model predictions," in *Proc. NIPS*, 2017.
[13] A. Ponce-Bobadilla *et al.*, "A practical guide to SHAP analysis in supervised machine learning," *Wiley Interdiscip. Rev. Data Min. Knowl. Discov.*, 2024.
[14] C. Tarabanis *et al.*, "Explainable SHAP-XGBoost models for in-hospital mortality prediction," *Digital Health*, 2023.
[15] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you? Explaining the predictions of any classifier," in *Proc. KDD*, 2016.
[16] C. Molnar, "Interpretable Machine Learning: LIME," online book chapter, 2019. [Online]. Available: https://christophm.github.io/interpretable-ml-book/lime.html